Proyecto 1

Simulacion de una Maquina de Turing determinista (1 cinta) para calcular Fibonacci

Curso:	Analisis y Diseno de Algoritmos
Entregable:	Informe tecnico + analisis empirico
Archivos:	maquina_turing.json, simulador_y_analisis_completo.ipynb
Fecha:	2026-02-28
Integrantes:	_____

Objetivo: evaluar la comprension del estudiante sobre notacion asintotica O usando el tiempo (pasos) de ejecucion de una Maquina de Turing simulada en Python.

1. Introduccion

Este proyecto implementa y analiza una Maquina de Turing determinista de una cinta que calcula el n-esimo numero de Fibonacci. La maquina se define en un archivo de configuracion (JSON) y un programa en Python simula su ejecucion mostrando configuraciones (estado, posicion de cabeza y contenido de cinta). Finalmente, se realiza un analisis empirico de tiempo de ejecucion (en milisegundos) y de numero de pasos (transiciones) en funcion del tamano de entrada.

Entregables cubiertos en este informe:

• Descripcion de convenciones de entrada/salida.

• Diagrama (alto nivel) de la maquina de Turing.

• Archivo con componentes de la maquina (JSON) y su estructura.

• Programa en Python que carga la maquina desde archivo, recibe entrada y lista configuraciones.

• Analisis empirico: entradas de prueba, dispersion de tiempos y regresion polinomial.

• Expresion del tiempo en notacion asintotica O.

2. Marco teorico

2.1 Notacion O y tiempo en Maquinas de Turing

En una Maquina de Turing (MT), el tiempo de ejecucion se mide como el numero de pasos o transiciones aplicadas antes de detenerse. Para una entrada de longitud n, la complejidad temporal se expresa como T(n)=numero de pasos en el peor caso. Usamos notacion O(·) para acotar el crecimiento asintotico de T(n).

2.2 Sucesion de Fibonacci

Se utiliza la definicion: F(0)=0, F(1)=1 y F(n)=F(n-1)+F(n-2) para n>=2. La sucesion crece aproximadamente como phi^n, donde phi≈1.618 (razon aurea).

3. Convenciones de representacion (Entregable 1)

Se adopta una codificacion unaria para la entrada y salida, y marcadores de trabajo en la cinta.

Elemento	Convencion
Entrada	n se representa como 1^n (n veces el simbolo '1'). Para n=0, la cinta inicia en blanco '_'.
Salida	Al detenerse, la maquina deja un separador '#' y a su derecha una cadena 0^F(n). El resultado se interpreta contando los '0' consecutivos a la derecha del primer '#'.
Marcadores	X marca un '1' ya procesado. x e y son marcadores temporales durante copias. H es un marcador temporal del separador durante la limpieza final. '_' representa blanco.

Ejemplo: n=5 -> entrada 11111, salida final #00000 (F(5)=5).

4. Diseno de la Maquina de Turing (Entregable 2)

La maquina opera en fases: inicializacion del area de trabajo, bucle principal (una iteracion por cada '1'), construccion del siguiente Fibonacci mediante copias en unario y limpieza final.

4.1 Diagrama de flujo (alto nivel)

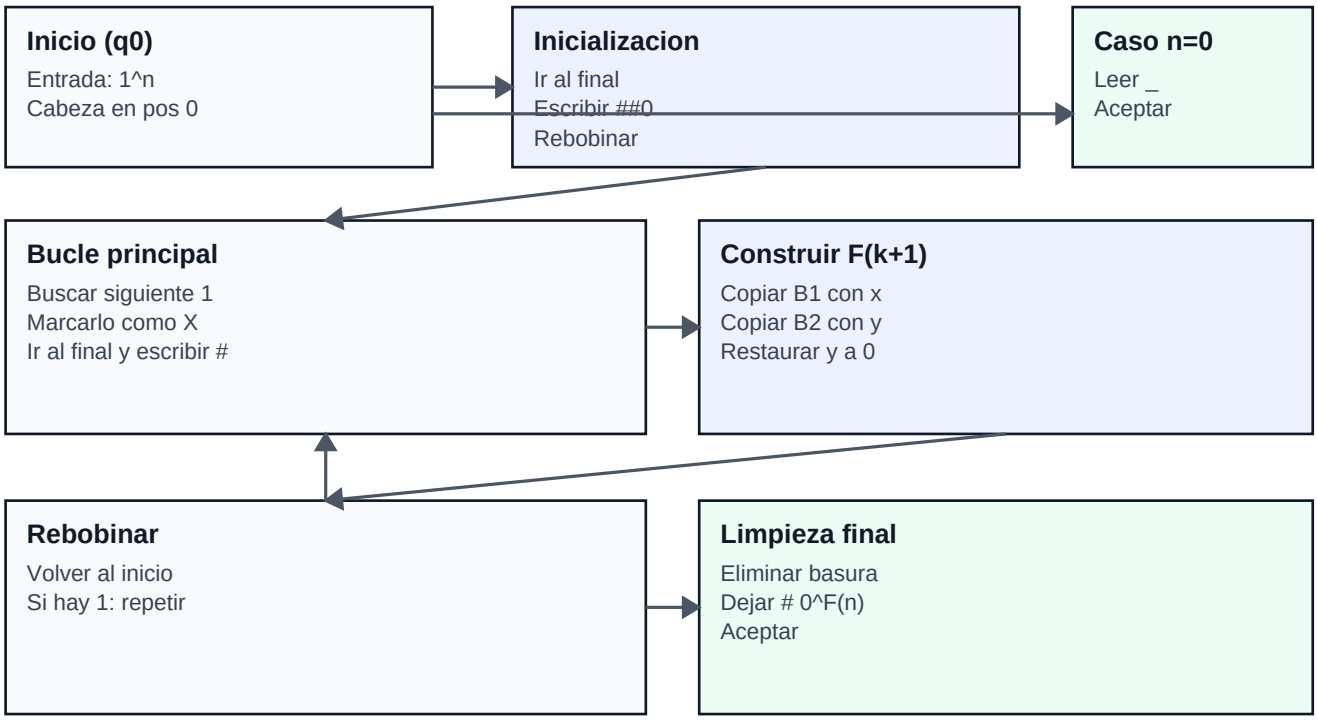

Figura 1. Flujo de alto nivel de estados y fases de la MT.

4.2 Componentes formales

Componente	Valor
Q (estados)	22 estados (ej.: q0, find_1, copy_B1, copy_B2, cleanup_erase, q_accept).
Sigma (entrada)	1
Gamma (cinta)	1, 0, X, #, x, y, H, _
Blanco	_
Estado inicial	q0
Aceptacion / rechazo	q_accept / q_reject
Delta (transiciones)	60 reglas δ(q,s) -> (q',s',mov).

4.3 Idea de computo (por que funciona)

El calculo de Fibonacci en unario se reduce a copia y concatenacion: para F(k+1)=F(k)+F(k-1), la maquina recorre y copia los bloques de ceros que representan F(k-1) y F(k) hacia la derecha. Los marcadores x e y evitan copias duplicadas y permiten regresar al punto correcto. Cada iteracion consume un '1' marcandolo como X.

5. Archivo de componentes (Entregable 3)

Los componentes de la MT se almacenan en un archivo JSON. El simulador puede ejecutar cualquier MT determinista de una cinta que respete el esquema.

5.1 Esquema del archivo JSON

Clave	Descripcion
estados	Lista de estados Q.
alfabeto_entrada	Alfabeto de entrada Sigma.
alfabeto_cinta	Alfabeto de cinta Gamma (incluye marcadores).
estado_inicial	Estado inicial q0.
estado_aceptacion	Estado de aceptacion.
estado_rechazo	Estado de rechazo.
transiciones	Diccionario: transiciones[q][s] = [q', s', 'L'/'R'].

El contenido completo del JSON se incluye en el Anexo A.

6. Programa en Python (Entregable 4)

El simulador aplica repetidamente la funcion de transicion δ. La cinta se representa como diccionario disperso: celdas ausentes se interpretan como blanco '_'.

6.1 Flujo de ejecucion del simulador

• Cargar JSON: leer estados, alfabetos, estados especiales y transiciones.

• Inicializar cinta con la entrada (1^n), cabeza=0 y estado=q0.

• Repetir: leer simbolo bajo la cabeza; buscar δ(estado, simbolo).

• Aplicar: escribir, cambiar estado y mover cabeza (L/R).

• Detener: q_accept o q_reject, o transicion indefinida.

6.2 Listado de configuraciones (requisito 4.c)

Cada configuracion incluye paso, estado, posicion de cabeza e instante de la cinta (corchetes [ ] marcan la cabeza).

```
Paso    0 | Estado=q0             | Cabeza=   0 | Cinta: [1] 1 1 1 1
Paso    1 | Estado=init_find_end  | Cabeza=   1 | Cinta: X [1] 1 1 1
Paso    2 | Estado=init_find_end  | Cabeza=   2 | Cinta: X 1 [1] 1 1
Paso    3 | Estado=init_find_end  | Cabeza=   3 | Cinta: X 1 1 [1] 1
Paso    4 | Estado=init_find_end  | Cabeza=   4 | Cinta: X 1 1 1 [1]
Paso    5 | Estado=init_find_end  | Cabeza=   5 | Cinta: X 1 1 1 1 [_]
Paso    6 | Estado=init_write_hash2 | Cabeza=   6 | Cinta: X 1 1 1 1 # [_]
Paso    7 | Estado=init_write_0   | Cabeza=   7 | Cinta: X 1 1 1 1 # # [_]
...
Paso  328 | Estado=cleanup_finish | Cabeza=  14 | Cinta: [_] _ _ H 0 0 0 0 0
Paso  329 | Estado=cleanup_finish | Cabeza=  15 | Cinta: [_] _ H 0 0 0 0 0
Paso  330 | Estado=cleanup_finish | Cabeza=  16 | Cinta: [_] H 0 0 0 0 0
Paso  331 | Estado=cleanup_finish | Cabeza=  17 | Cinta: [H] 0 0 0 0 0
HALT | Estado final=q_accept | Pasos=332 | Cinta: # [0] 0 0 0 0
```

Figura 2. Extracto de configuraciones para n=5 (entrada 11111).

6.3 Fragmento clave del codigo (bucle principal)

```
while state not in (q_accept, q_reject):
    sym = read(tape, head)
    if (state, sym) not in delta:
        state = q_reject
        break
    new_state, write_sym, move = delta[state][sym]
    write(tape, head, write_sym)
    state = new_state
    head = head + 1 if move == 'R' else head - 1
    steps += 1
```

El codigo completo utilizado para simulacion y analisis se entrega en el notebook **simulador_y_analisis_completo.ipynb**. El Anexo B incluye un simulador consolidado.

7. Analisis empirico (Entregable 5)

Se ejecuta la MT para entradas 1^n. Cada n se ejecuta 30 veces y se reporta la mediana del tiempo (ms). Tambien se registra el numero de pasos (transiciones), que define el tiempo teorico en MT.

7.1 Entradas de prueba (5.a)

n	Entrada (unario)	|entrada|	F(n)	Pasos	Tiempo mediana (ms)
1	1	1	1	21	0.009
2	11	2	1	54	0.022
3	111	3	2	109	0.047
4	1111	4	3	191	0.069
5	11111	5	5	332	0.131
6	111111	6	8	589	0.222
7	1111111	7	13	1115	0.414
8	11111111	8	21	2276	0.816
9	111111111	9	34	5011	1.823
10	1111111111	10	55	11713	4.347
11	11111111111	11	89	28574	10.644
12	111111111111	12	144	71655	27.388

7.2 Dispersion de tiempos (5.b)

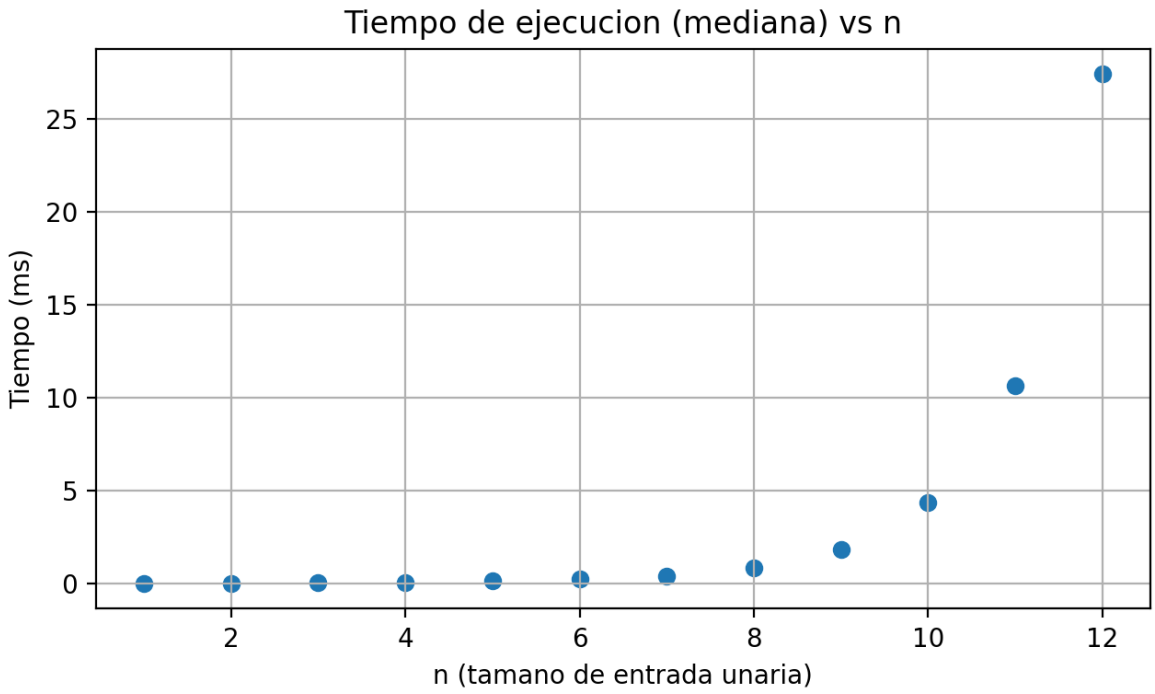

Figura 3. Tiempo de ejecucion (mediana) vs n.

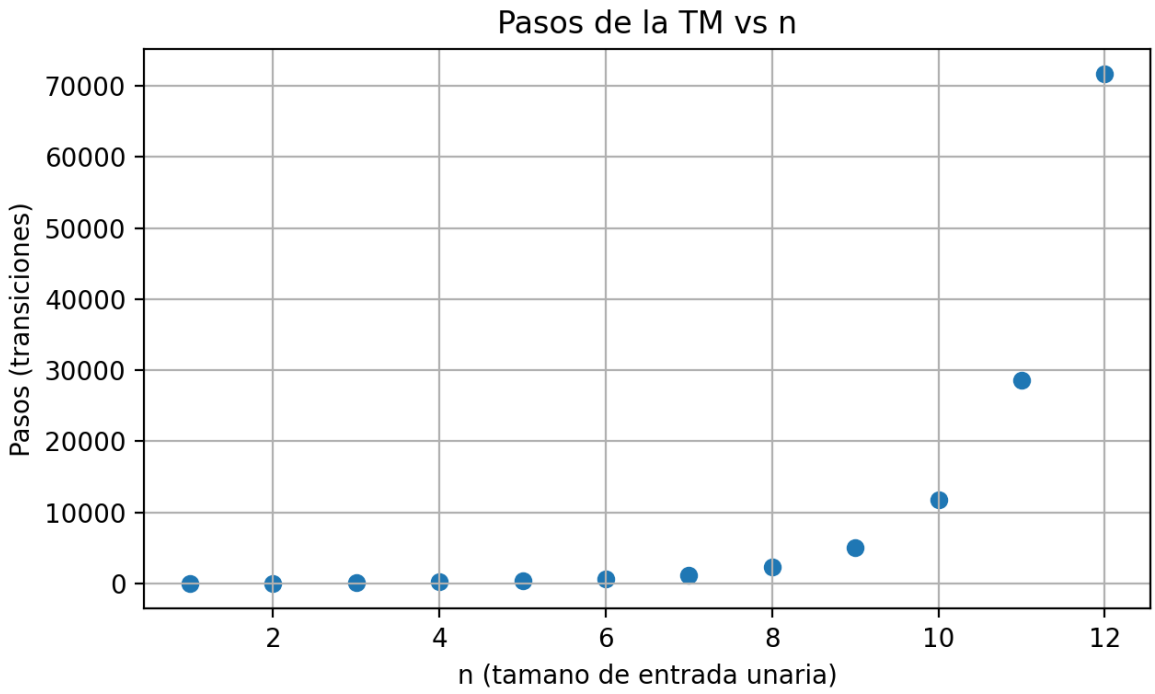

Figura 4. Pasos de la MT vs n.

7.3 Regresion polinomial (5.c)

Se ajusta una regresion polinomial a los datos. Para tiempo (ms) vs n se selecciono grado 6 con R^2(test)= 0.913. Para pasos vs n se selecciono grado 6 con R^2(test)= 0.931.

Tiempo (ms): T(n) ≈ 3.912e-04*n^6 + -1.250e-02*n^5 + 1.578e-01*n^4 + 2.285e+00 + ...

Pasos: S(n) ≈ 9.355e-01*n^6 + -2.960e+01*n^5 + 3.707e+02*n^4 + 5.245e+03 + ...

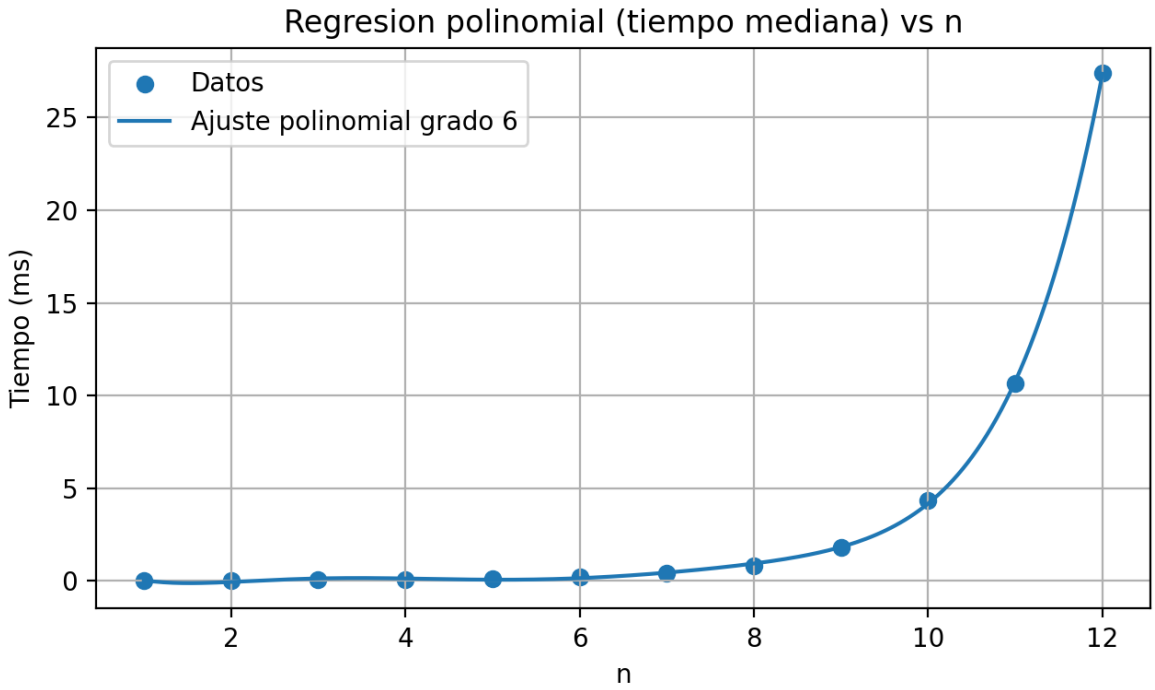

Figura 5. Ajuste polinomial para tiempo (mediana) vs n.

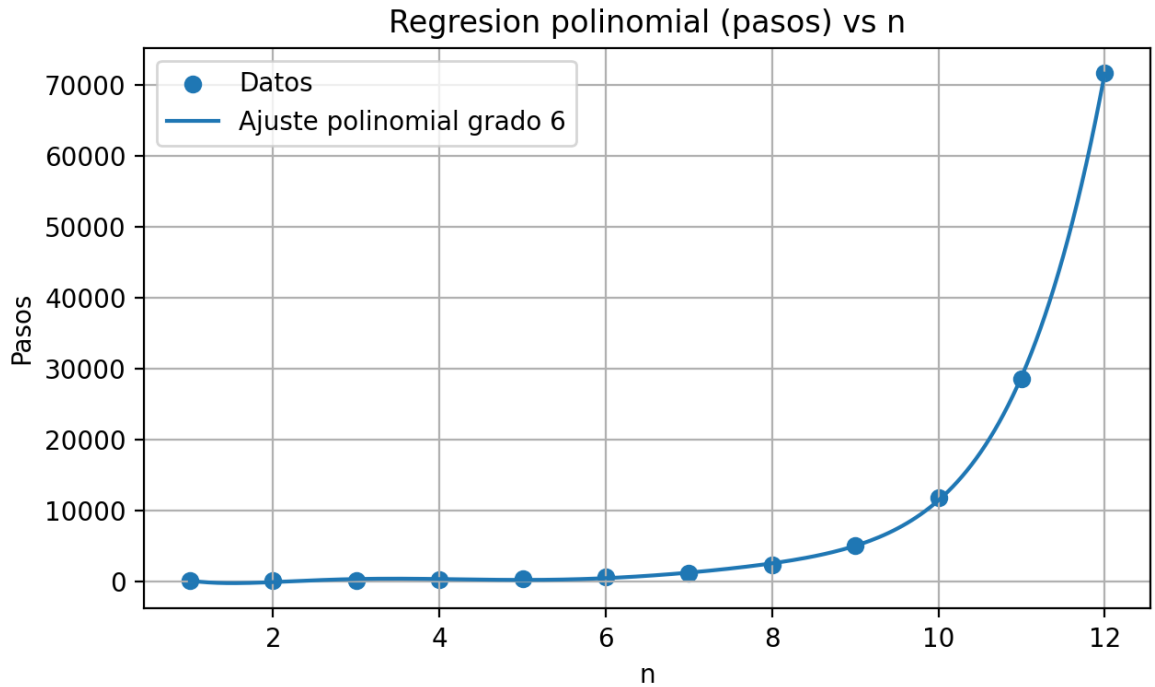

Figura 6. Ajuste polinomial para pasos vs n.

7.4 Expresion en notacion asintotica O

Aunque la regresion polinomial ajusta los datos en el rango observado, para complejidad asintotica en MT es natural razonar con pasos y tamano de salida. La MT construye Fibonacci por copias de bloques en unario.

Empiricamente, el cociente pasos/(F(n)^2) cae en el rango [3.46, 54.00] (mediana 7.90). Esto sugiere crecimiento cercano a Θ(F(n)^2) en pasos.

Dado que F(n) ~ phi^n, entonces F(n)^2 ~ (phi^2)^n ≈ 2.618^n. Por tanto, una expresion razonable es:

S(n) = O(F(n)^2) = O((phi^2)^n) (phi≈1.618).

8. Conclusiones

• Se definio una MT determinista de una cinta para Fibonacci con convenciones claras de entrada y salida.

• El simulador en Python carga los componentes desde JSON y lista configuraciones completas (estado, cabeza y cinta).

• El analisis empirico muestra crecimiento rapido de pasos y tiempo al aumentar n, coherente con salida en unario y copias repetidas.

• Se cumple regresion polinomial y se propone O(F(n)^2) ≈ O(2.618^n) como expresion asintotica basada en pasos.

Referencias

• Wikipedia. 'Sucesion de Fibonacci'. https://es.wikipedia.org/wiki/Sucesi%C3%B3n_de_Fibonacci

• Sipser, Michael. Introduction to the Theory of Computation. (Maquina de Turing y complejidad temporal).

Anexo A. maquina_turing.json (contenido completo)

Se incluye el archivo JSON con los componentes y transiciones de la MT.


```
{
  "estados": [
    "q0",
    "init_find_end",
    "init_write_hash2",
    "init_write_0",
    "rewind",
    "find_1",
    "goto_end_for_hash",
    "find_B1_start",
    "skip_B1",
    "copy_B1",
    "B1_goto_end",
    "B1_return",
    "copy_B2",
    "B2_goto_end",
    "B2_return",
    "restore_B2",
    "cleanup_goto_end",
    "cleanup_skip_ans",
    "cleanup_erase",
    "cleanup_finish",
    "q_accept",
    "q_reject"
  ],
  "alfabeto_entrada": [
    "1"
  ],
  "alfabeto_cinta": [
    "1",
    "0",
    "X",
    "#",
    "x",
    "y",
    "H",
    "_"
  ],
  "estado_inicial": "q0",
  "estado_aceptacion": "q_accept",
  "estado_rechazo": "q_reject",
  "transiciones": {
    "q0": {
      "1": [
        "init_find_end",
        "X",
        "R"
      ],
      "_": [
        "q_accept",
        "#",
        "R"
      ]
    },
    "init_find_end": {
```

```
      "1": [
        "init_find_end",
        "1",
        "R"
      ],
      "_": [
        "init_write_hash2",
        "#",
        "R"
      ]
    },
    "init_write_hash2": {
      "_": [
        "init_write_0",
        "#",
        "R"
      ]
    },
    "init_write_0": {
      "_": [
        "rewind",
        "0",
        "L"
      ]
    },
    "rewind": {
      "0": [
        "rewind",
        "0",
        "L"
      ],
      "#": [
        "rewind",
        "#",
        "L"
      ],
      "x": [
        "rewind",
        "x",
        "L"
      ],
      "y": [
        "rewind",
        "y",
        "L"
      ],
      "X": [
        "rewind",
        "X",
        "L"
      ],
      "1": [
        "rewind",
        "1",
        "L"
```

```
      ],
      "_": [
        "find_1",
        "_",
        "R"
      ]
    },
    "find_1": {
      "X": [
        "find_1",
        "X",
        "R"
      ],
      "1": [
        "goto_end_for_hash",
        "X",
        "R"
      ],
      "#": [
        "cleanup_goto_end",
        "#",
        "R"
      ]
    },
    "goto_end_for_hash": {
      "1": [
        "goto_end_for_hash",
        "1",
        "R"
      ],
      "#": [
        "goto_end_for_hash",
        "#",
        "R"
      ],
      "0": [
        "goto_end_for_hash",
        "0",
        "R"
      ],
      "x": [
        "goto_end_for_hash",
        "x",
        "R"
      ],
      "X": [
        "goto_end_for_hash",
        "X",
        "R"
      ],
      "_": [
        "find_B1_start",
        "#",
        "L"
      ]
```

```
    },
    "find_B1_start": {
      "0": [
        "find_B1_start",
        "0",
        "L"
      ],
      "#": [
        "skip_B1",
        "#",
        "L"
      ]
    },
    "skip_B1": {
      "0": [
        "skip_B1",
        "0",
        "L"
      ],
      "#": [
        "copy_B1",
        "#",
        "R"
      ],
      "x": [
        "copy_B1",
        "x",
        "R"
      ]
    },
    "copy_B1": {
      "0": [
        "B1_goto_end",
        "x",
        "R"
      ],
      "#": [
        "copy_B2",
        "#",
        "R"
      ]
    },
    "B1_goto_end": {
      "0": [
        "B1_goto_end",
        "0",
        "R"
      ],
      "#": [
        "B1_goto_end",
        "#",
        "R"
      ],
      "_": [
        "B1_return",
```

```
        "0",
        "L"
      ]
    },
    "B1_return": {
      "0": [
        "B1_return",
        "0",
        "L"
      ],
      "#": [
        "B1_return",
        "#",
        "L"
      ],
      "x": [
        "copy_B1",
        "x",
        "R"
      ]
    },
    "copy_B2": {
      "0": [
        "B2_goto_end",
        "y",
        "R"
      ],
      "#": [
        "restore_B2",
        "#",
        "L"
      ]
    },
    "B2_goto_end": {
      "0": [
        "B2_goto_end",
        "0",
        "R"
      ],
      "#": [
        "B2_goto_end",
        "#",
        "R"
      ],
      "_": [
        "B2_return",
        "0",
        "L"
      ]
    },
    "B2_return": {
      "0": [
        "B2_return",
        "0",
        "L"
```

```
      ],
      "#": [
        "B2_return",
        "#",
        "L"
      ],
      "y": [
        "copy_B2",
        "y",
        "R"
      ]
    },
    "restore_B2": {
      "y": [
        "restore_B2",
        "0",
        "L"
      ],
      "#": [
        "rewind",
        "#",
        "L"
      ]
    },
    "cleanup_goto_end": {
      "#": [
        "cleanup_goto_end",
        "#",
        "R"
      ],
      "x": [
        "cleanup_goto_end",
        "x",
        "R"
      ],
      "0": [
        "cleanup_goto_end",
        "0",
        "R"
      ],
      "_": [
        "cleanup_skip_ans",
        "_",
        "L"
      ]
    },
    "cleanup_skip_ans": {
      "0": [
        "cleanup_skip_ans",
        "0",
        "L"
      ],
      "#": [
        "cleanup_erase",
        "H",
```

```
        "L"
      ]
    },
    "cleanup_erase": {
      "0": [
        "cleanup_erase",
        "_",
        "L"
      ],
      "x": [
        "cleanup_erase",
        "_",
        "L"
      ],
      "y": [
        "cleanup_erase",
        "_",
        "L"
      ],
      "#": [
        "cleanup_erase",
        "_",
        "L"
      ],
      "X": [
        "cleanup_erase",
        "_",
        "L"
      ],
      "1": [
        "cleanup_erase",
        "_",
        "L"
      ],
      "_": [
        "cleanup_finish",
        "_",
        "R"
      ]
    },
    "cleanup_finish": {
      "_": [
        "cleanup_finish",
        "_",
        "R"
      ],
      "H": [
        "q_accept",
        "#",
        "R"
      ]
    }
  }
}
```

Anexo B. Simulador en Python (codigo consolidado)

```
import json
import time

class TuringMachine:
    def __init__(self, json_path):
        cfg = json.load(open(json_path, "r", encoding="utf-8"))
        self.q0 = cfg["estado_inicial"]
        self.q_accept = cfg["estado_aceptacion"]
        self.q_reject = cfg["estado_rechazo"]
        self.blank = "_"
        self.sigma = set(cfg["alfabeto_entrada"])
        self.delta = cfg["transiciones"]

    def _read(self, tape, pos):
        return tape.get(pos, self.blank)

    def _write(self, tape, pos, sym):
        if sym == self.blank:
            tape.pop(pos, None)
        else:
            tape[pos] = sym

    def run(self, input_str, verbose=True, max_steps=2_000_000):
        # validar entrada
        for ch in input_str:
            if ch not in self.sigma:
                raise ValueError("Simbolo invalido en entrada")

        tape = {i: ch for i, ch in enumerate(input_str)}  # cinta dispersa
        head = 0
        state = self.q0
        steps = 0

        def format_tape():
            if tape:
                mn = min(min(tape.keys()), head)
                mx = max(max(tape.keys()), head)
            else:
                mn = mx = head
            out = []
            for j in range(mn, mx + 1):
                s = self._read(tape, j)
                out.append(f"[{s}]" if j == head else s)
            return " ".join(out)

        while state not in (self.q_accept, self.q_reject):
            sym = self._read(tape, head)
            if verbose:
                print(f"Paso {steps:6d} | Estado={state:14s} | Cabeza={head:6d} | Cinta: {format_tape()}")

            try:
                new_state, write_sym, move = self.delta[state][sym]
            except KeyError:
                state = self.q_reject
                break
```

```
            self._write(tape, head, write_sym)
            state = new_state
            head += 1 if move == "R" else -1

            steps += 1
            if steps > max_steps:
                state = self.q_reject
                break

        return (state == self.q_accept, state, steps, tape, head)

def extract_output_fib(tape):
    # Convencion: contar '0' a la derecha del primer '#'
    if not tape:
        return 0
    mn, mx = min(tape.keys()), max(tape.keys())
    s = "".join(tape.get(i, "_") for i in range(mn, mx + 1))
    if "#" not in s:
        return 0
    right = s.split("#", 1)[1]
    c = 0
    for ch in right:
        if ch == "0":
            c += 1
        elif ch == "_":
            break
        else:
            break
    return c

# Ejemplo de uso:
# tm = TuringMachine("maquina_turing.json")
# ok, st, steps, tape, head = tm.run("11111", verbose=True)
# print("F(n) =", extract_output_fib(tape))
```